Mid-1 Solution

Q1: Short Answers

i. Primary vs Secondary Cache

Primary (L1) cache is built into the CPU, very fast and small, holding immediate-use data. Secondary (L2) cache is larger, slightly slower, and backs up L1 by storing additional frequently used data.

ii. Purpose of Listing File

It shows the source code with corresponding machine code and memory addresses, helping verify assembly and debug errors.

iii. Directive vs Instruction

A directive tells the assembler how to process code (no machine code generated), while an instruction tells the CPU what operation to perform (generates executable machine code).

Example: .DATA is a directive; MOV AL, 5 is an instruction.

iv. Why Data Transfer Instructions Don't Update Flags

They only copy data without performing computation, so there is no arithmetic/logical result for flags to reflect.

v. Purpose of EAX, ECX, EDX

EAX is used for arithmetic operations and return values; ECX acts as a loop/shift counter; EDX assists EAX in multiplication/division and I/O operations.

Q2 (a).I: Memory Map with Physical Addresses

Physical Address = (DS x 10h) + Offset = (0C40h x 10h) + Offset = 0C400h + Offset

Starting offset = ABCDh -> Starting physical address = 0C400h + ABCDh = 16FCDh

Data is stored little-endian (least significant byte at the lowest address).

Arr1 (DWORD 11E2CD44h, 55F677ABh) - 8 bytes

Offset	Physical Address	Value	Description
ABCDh	16FCDh	44h	Arr1[0] LSB
ABCEh	16FCEh	CDh	
ABCFh	16FCFh	E2h	
ABD0h	16FD0h	11h	Arr1[0] MSB
ABD1h	16FD1h	ABh	Arr1[1] LSB
ABD2h	16FD2h	77h	
ABD3h	16FD3h	F6h	
ABD4h	16FD4h	55h	Arr1[1] MSB

Arr2 (SBYTE 11h, -27, 33h, COUNT DUP(44h)) - 6 bytes (-27 = E5h)

Offset	Physical Address	Value	Description
ABD5h	16FD5h	11h	Arr2[0]
ABD6h	16FD6h	E5h	Arr2[1] = -27
ABD7h	16FD7h	33h	Arr2[2]
ABD8h	16FD8h	44h	DUP #1
ABD9h	16FD9h	44h	DUP #2

ABDAh	16FDAh	44h	DUP #3

Arr3 (WORD 34h, 0A678h, 9ABCh) - 6 bytes

Offset	Physical Address	Value	Description
ABDBh	16FDBh	34h	Arr3[0] LSB
ABDCh	16FDCh	00h	Arr3[0] MSB
ABDDh	16FDDh	78h	Arr3[1] LSB
ABDEh	16FDEh	A6h	Arr3[1] MSB
ABDFh	16FDFh	BCh	Arr3[2] LSB
ABE0h	16FE0h	9Ah	Arr3[2] MSB

Total: 20 bytes, offsets ABCDh-ABE0h, physical addresses 16FCDh-16FE0h.

Q2 (a).II: Instruction-by-Instruction Results

#	Instruction	Result
a	MOV AX, [Arr3+2]	AX = A678h
b	MOVSX BX, BYTE PTR [Arr2+4]	BX = 0044h
c	SUB AX,BX	AX = A634h, ZF=0, SF=1, OF=0, PF=0
d	MOV CX, SIZEOF Arr2	CX = 0006h
e	MOV CH, BYTE PTR [Arr3+2]	CH = 78h (CX = 7806h)
f	DEC CX	CX = 7805h, CF=0 (unaffected), PF=1, ZF=0, SF=0
g	MOV EDX, DWORD PTR [Arr1+4]	EDX = 55F677ABh
h	ADD AX,0Fh	AX = A643h, ZF=0, SF=1, CF=0, PF=0

Q3: Copy DWORD array Dval into BYTE array Bval

Program using BYTE PTR and LOOP to copy every byte of Dval into Bval:

```
.DATA
Dval DWORD 12345678h, 0ABCDEF12h
Bval BYTE SIZEOF Dval DUP(?)

.CODE
main PROC
    MOV ECX, SIZEOF Dval      ; ECX = 8 (loop counter = number of bytes)
    MOV ESI, 0                ; index into Dval (source)
    MOV EDI, 0                ; index into Bval (destination)

COPY_LOOP:
    MOV AL, BYTE PTR [Dval + ESI]   ; get one byte from Dval
    MOV BYTE PTR [Bval + EDI], AL   ; store it into Bval
    INC ESI                          ; move to next source byte
    INC EDI                          ; move to next destination byte
    LOOP COPY_LOOP                   ; decrement ECX, repeat until ECX = 0

    INVOKE ExitProcess, 0
main ENDP
END main
```